

Empowering Real-Time Communication: A Seamless Chatting System Using Websocket



Kaiwalya Deshpande, Ayush Jain, Abhinav, Sushruta Mishra, Shalini Goel, and Rachit Garg

Abstract Real-time chatting systems are at an all-time high in terms of popularity. They allow the users to communicate among themselves instantly. The aim of this paper is to present a solution to implement a real-time chatting system in a feasible and applicable environment. This paper utilizes components such as Websockets using socket.io in Node.js. Websockets are a modern technology which enables the server and client to establish a continuous connection for real-time communication, and socket.io is a JavaScript library which provides a high-level abstraction of Websockets. In conclusion, this research explores the vast features of Websocket technology in the field of real-time communication. This paper advocates for the widespread adoption of Websocket-based chatting systems, bringing closer the new era of seamless interaction in the modern age.

Keywords Websockets · Real-time · Chatting · Socket.io · Node.js

K. Deshpande · A. Jain · Abhinav · S. Mishra (✉)

Kalinga Institute of Industrial Technology, Deemed to be University, Bhubaneswar, India

e-mail: sushruta.mishrafcs@kiit.ac.in

K. Deshpande

e-mail: Kaiwalya@live.in

A. Jain

e-mail: 20051827@kiit.ac.in

S. Goel

Department of Information Communication and Technology (ICT), Technia Institute of Technology and Management, Delhi, India

R. Garg

School of Computer Science and Engineering, Lovely Professional University, Bengaluru, India

1 Introduction

In the new era, real-time messaging has become an integral part of our daily life. Real-time chatting has become one of the most widely used media of communication. These have provided the luxury of sharing information instantly.

The demand for these real-time interaction systems is at an all-time high; hence, the objective of this paper is to implement a system which presents and explores the possibilities provided by Websockets, a modern communication protocol, specifically focusing on the “socket.io” library. socket.io is a JavaScript library which makes it easier to integrate Websockets inside the Node.js framework.

1.1 Background and Significance

Communication has been hindered for a long time due to the inability to provide the information with fluidity. The communications between multiple users were also a huge hindrance since broadcasting information from all ends was not feasible. Real-time chatting systems emerged to bridge the gap, and it enabled the users to exchange information almost instantaneously and seamlessly [1]. Since then, the systems have seen very diverse user applications including online gaming, customer support, collaborative environments, etc.

1.2 Aim of the Paper

The primary aim of this paper is to explore all the possibilities of real-time chatting systems. By using socket.io, the Websocket can be implemented in the JavaScript. The library establishes a persistent connection between servers and clients through which the communication between either two or multiple users takes place. The goal is to present a well-rounded implementation which is also practically feasible and applicable in real world [2].

1.3 Overview of Websockets and Socket.io

Websockets are a web-based protocol which allows bidirectional information transfer between the server and the client; this connection enables the real-time data transfer from both ends. The library “socket.io” is used to implement the Websockets; it simplifies the implementation of Websockets such that all the capabilities of the protocol are directly accessible in the Node.js framework.

1.4 Contribution of the Research

This research contributes in exploring the real-time communication field by providing a sophisticated approach of implementing a real-time chatting system using Websockets through socket.io. This research contributes to wide variety of solution of seamless interactions catering to the ever-evolving need of digital communication.

The following sections will explore the intricacies of Websockets and socket.io, detailing all the methodologies used in building the real-time chat system involving the security measures taken to keep the user data secure to actually make the real-time chat system feasible of real-world usage [3].

This research contributes in exploring the real-time communication field by providing a sophisticated approach of implementing a real-time chatting system using Websockets through socket.io. This research contributes to wide variety of solution of seamless interactions catering to the ever-evolving need of digital communication.

This paper contributes in the field of real-time communications with minimal delay using the Websockets capabilities to make use of the servers' capabilities of sending a response to the client without any additional prompt from the client.

Exploring the socket.io library in Node.js to implement the Websockets, this approach shows the usefulness of the library and the power of the Websockets protocol, enabling everyone to build upon this idea to further improve the applications currently in circulation in a wide range of features ranging from quality of life features such as notifications to essential features such as real-time stock updates.

The following sections will explore the intricacies of Websockets and socket.io, detailing all the methodologies used in building the real-time chat system involving the security measures taken to keep the user data secure to actually make the real-time chat system feasible of real-world usage.

2 Literature Review

2.1 The Need for Real-Time Communication

The client–server model has existed for as long as the birth of web-based architecture, clients send a request for a resource to the server, and the server processes the request and responds with the resources if available. This model has been the roots of internet for the longest time; however, it has become increasingly outdated and lackluster for the various demands of modern application. The applications are evolving and becoming more and more dynamic and interactive; the need for servers to push responses to clients at will is increasing which this model does not provide. This need for bidirectional communication at unpredictable intervals gave rise to the protocols such as Websockets and HTTP Polling [4].

2.2 HTTP Polling

An early solution to the lack of the ability of a server to initiate communication was HTTP Polling. This model has been around for over two decades [5]. In this mode, the client sends frequent requests over a periodic time for the lifetime of the connection and the server responds with empty or baseline response until an adequate response is generated. The latency of this model is the time gap between each request from the client. The keep-alive header provides some much-needed optimization for this model, and it reuses the same TCP connection for subsequent requests from the client heavily reducing the overhead of creating a new TCP connection for each request [6, 7].

This model creates a facade of bidirectional communication; the server is still not able to voluntarily send updates. For longer duration of time, this model ends up wasting significant amount of network traffic along with all the unnecessary requests made by the client. In addition to all this, updates from the server can be pushed with the same frequency with which client makes requests [8].

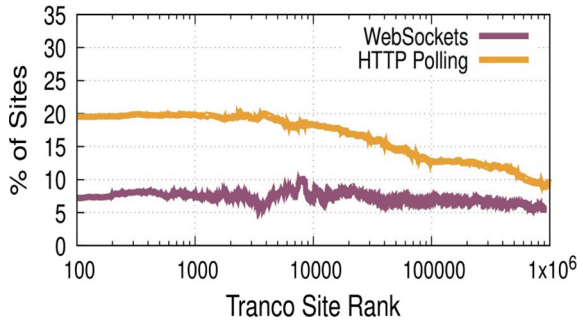
2.3 HTTP Long Polling

An optimization available on polling makes use of the ability of server to hold off from responding to client requests immediately [1]. In this model, the server receives a request from client and chooses to not respond to it until the data are available or the connection timeouts; on the occurrence of a timeout, the client re-sends the same request and the server continues to wait for the data. This method does reduce the load on the network, but the issue of inconsistency and unpredictability of a response from server still persists.

2.4 Websockets

The WebSocket protocol offers a significant improvement over previous real-time mechanism. It offers a full-duplex communication between client and server allowing both parties to send and receive data simultaneously. This is facilitated by establishing a persistent TCP connection between the two endpoints which in turn is used to exchange lightweight WebSocket frames. Websocket are mostly used in either small, frequent data exchanges or server-initiated communication. A WebSocket connection is initiated by a client sending a HTTP GET request with the header Upgrade: WebSocket. If the server supports WebSocket, it will respond with HTTP 101: Switching Protocols, and once the connection is established, both the client and server can send and receive data frames at will [9]. WebSocket also offers a number of benefits over traditional HTTP-based real-time update mechanisms, namely: Lower

Fig. 1 Cumulative frequency of Websocket and HTTP polling use over the Tranco top 1 M



latency: Websocket avoids the issues of HTTP polling resulting in lower latency and more responsive applications. Reduced bandwidth usage: A single TCP connection is established throughout the data exchange which reduces bandwidth usage. Figure 1 shows the Websocket cumulative frequency and HTTP polling.

2.5 Web-Server Implementation

Along with bidirectional connection between client and server for real-time chatting, a solution for managing database library of users and messages data is also needed.

- (1) *PHP and MySQL*: Some previous generation chatting application like Yahoo messenger used PHP and MySQL to manage the servers and handle client requests [10]. PHP is a HTML-enabled, server-side utility, and MySQL is an open-source framework for managing relational databases, created and maintained by MySQL. PHP and MySQL have a good synergy; however, due to the synchronous nature of this solution, it is very time-consuming to operate on a large database [11].
- (2) *jQuery*: jQuery is a JavaScript-based design tool; it employs Document Object Model a.k.a. DOM to explore documents. Developers also utilize the jQuery to build plugins which work from JavaScript libraries [6]. Along with this, the modular nature of jQuery also eases the process of building complex and dynamic websites and web pages.
- (3) *MERN Stack*: For building fast and highly responsive web pages and web apps, Node.js is a great tool [12]. It can deliver data to or from a web server very efficiently. For the longest time, the inability of CPU to process multiple tasks at a time was a big roadblock for faster computation time. With the introduction of multi-threading, Node.js was optimized to work by running separate event-driven threads.

MongoDB is a cross-platform document-oriented database engine. It is a Nonstructured Query Language a.k.a NoSQL. It uses JSON-like data storing structures with optional schemas.

All of these four very powerful frameworks, MongoDB, Express.js, React.js, and Node.js, in conjunction create a very easy to build and faster to use web page and websites designing tool.

3 System Design

3.1 Servers

In the real-time chat implementation designed for this research, two servers are built and maintained:

- Client Side—this server is responsible for the user side access. This is what will be visible to the users, and they will be able to log into their accounts and send messages to other users directly or to a predefined group of users [13].
- Server Side—this server is responsible for the server-side management. All the API requests will be received and processed by the server. It will also be responsible to read and write the data into the MongoDB database.

A sample demonstration of client server communication is depicted in Fig. 2.

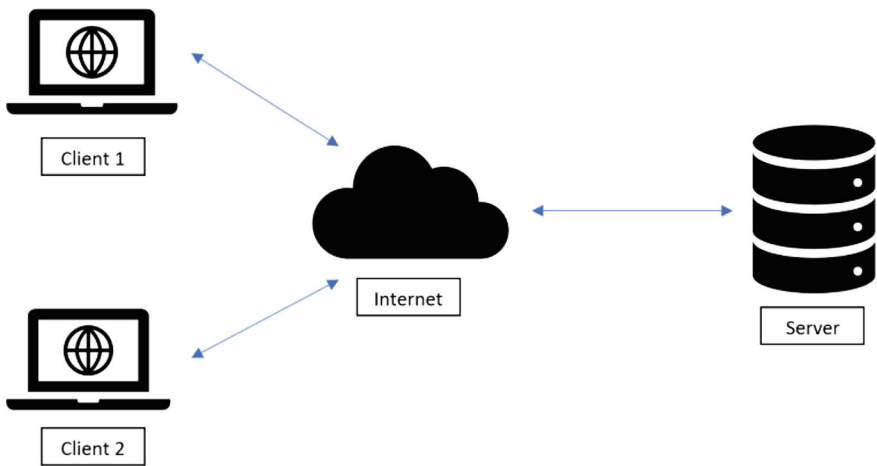


Fig. 2 Server client communication

Table 1 User schema model

S. No.	Field name	Data type	Explanation
1	Id	ObjectId	Unique id
2	Name	String	Name
3	Email	String	Email address
4	Password	String	Encrypted password
5	Pic	String	URL of picture

3.2 Authentication

All the user data are stored in a MongoDB database. The passwords of the users are stored in an encrypted format using “bcrypt” function provided by JavaScript. The schema of data stored of each user is shown in Table 1.

- (1) *Sign Up*: A new user can register their new account into the database by providing all the required details. The details are first validated to be unique (email address), and then the user is logged into their new account.
- (2) *Log in*: An already existing user can directly login from the homepage; they will require their unique email address and password [14]. Entered password will be encrypted and then compared with the stored password pulled from the MongoDB database.

3.3 Workflow

The workflow of the implementation is as follows:

- A user dubbed as Client 1 and another user dubbed as Client 2 access the portal through client-side server.
- They log into their respective accounts and open a chat.
- A room is created in socket.io where both of them are logged in.
- Client 1 sends a message from their client-side portal.
- The message along with the sender details is sent to the socket.io room.
- Inside the room the, receiver’s info is validated and the message is forwarded to the client portal of Client 2.
- Along with this, the back-end server also logs the message with sender and receiver details into the MongoDB database for persistent storage.
- Once both the clients leave the portal, the socket.io room is destroyed and the connection is discontinued.

React.js (front end) and Node.js/Express.js (back end) are both the frameworks of JavaScript which enable both the front end and the back end to be scripted in the same language. For storing the data into MongoDB, JavaScript Object Notation a.k.a. JSON files are used. These can directly interact with the JavaScript frameworks

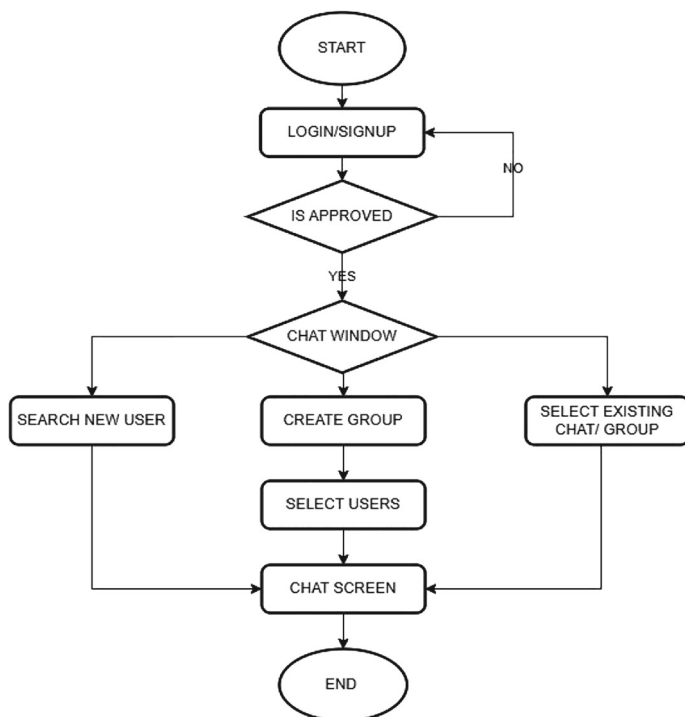


Fig. 3 Flowchart of system workflow

and thus it provides a very good option for quick and easy interactions. The overall workflow model is shown in Fig. 3.

4 Implementation and Result

The real-time application built for this research has shown the capabilities of Websocket protocol to an outstanding degree. The real-time chat system managed to implement what it originally set out to, as shown in Fig. 4. Along with this, a group feature has also been implemented where not just two users, but multiple can chat with everyone all in real time.

Along with this, there are many different functionalities that the chatting system is capable of. Some of these functionalities are listed as follows:

- The user can search for other people in the network to chat with by searching for their name or email address.
- The user can create a group, rename an existing group, or even add/remove members from the group.

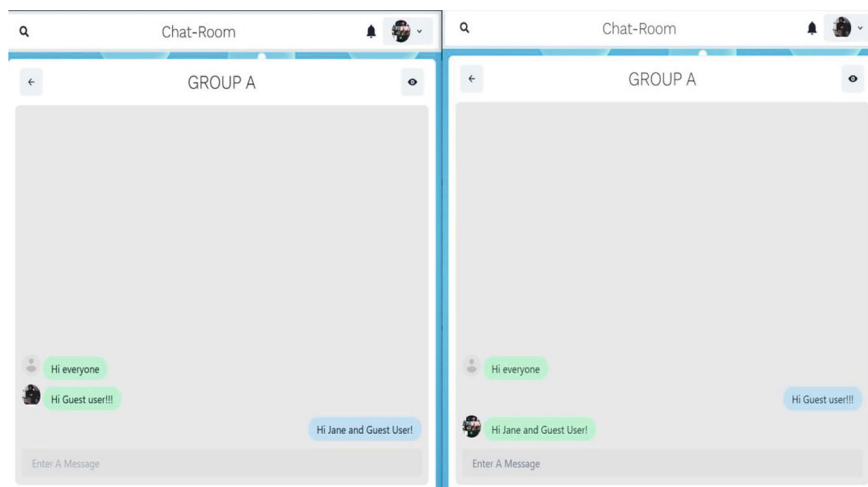


Fig. 4 Showcasing chatting in a group

- The add/remove group functionality is specific for the admins of the group.
- The user can also look up the details of the members of a group or of a particular user.
- Using socket.io, the user is also able to see if some other user is typing a message in the chat.
- A notification system is also implemented where if a user sends a message to another user, who is not currently in same chat, and then the other user will be notified about a new message in real-time through the use of socket.io.

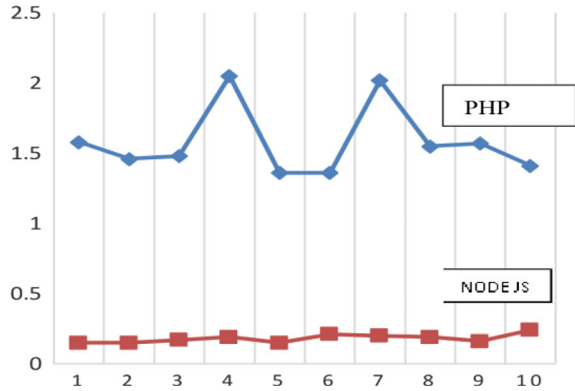
All of this was accomplished using MERN stack and the Websocket protocol wherever possible. The consistency and latency of the shared messages were near instantaneous and the other user received a message update from the server with zero input. The server was able to directly push messages to another user if they received any messages. This real-time chatting system was successful in demonstrating the capabilities of Websocket and hence could be considered a success.

5 Case Study and Applications

A similar study was also conducted where the authors created two applications, one using PHP and other using Node.js to find out which framework works better for such kind of real-time chatting systems [8]. It was shown also in the following Fig. 5, that Node.js overshadowed PHP by a landslide [15].

There are many such applications which are currently in the market that have successfully utilized WebSocket, some of them are as follows:

Fig. 5 Response time versus no. of requests graph



- **Google Docs:** It is a SaaS technology which allows users to create, edit, and share documents in real time [16]. In order to provide this real-time collaborative functionality, Google Docs uses Websockets, like the ability to be able to see what changes the other users are making in real-time.

A simplified overview of Websocket implementation in Google Docs is as follows:

- A Websocket connection to the Google Docs server is established as soon as a user logs into a Google Docs document.
- A snapshot of the document is shared with the user.
- The snapshot is presented to the user as editable content.
- The user begins editing the snapshot.
- While editing, the browser notes down the changed occurred and shares the data with Google Docs server.
- The servers update the notified changes into their stored document and then push the new snapshot onto all the connected users.
- After receiving a new snapshot from the server, all the browsers reflect the changes into their snapshots.
- This process is repeated until all the connected users stop editing the document.
- **Twitch:** It is a live video streaming platform which lets the users to either broadcast or view live video games or creative content or other forms of entertainment. It uses Websocket to provide the real-time video streaming and chat functionality. A simplified overview of Websocket implementation in Twitch is as follows:
 - A Websocket connection is established when a user opens a particular Twitch stream in their browser.
 - A stream of video frames is pushed from the Twitch stream to the browser.
 - The browser displays the frames.
 - Chatting service is also accessible to the users.
 - The twitch server distributes the messages to all connected users.

- **Stock Trading Platforms:** A lot of stock trading platforms utilize the Websocket to provide real-time stock quotations and order execution. This enables the users to respond to market stock changes in the most informed manner. A simplified overview of Websocket implementation in stock trading platform is as follows:
 - A trader, through their browser, opens the stock trading website/web app. The browser establishes a Websocket connection to the back-end server.
 - All the latest updates about the trading market in real-time are pushed from the server to the users' browser.
 - The browser receives and displays the latest updates.
 - The user chooses to either buy or sell a stock.
 - The browser sends a purchase/sell order request to the servers.
 - The trading platform's server executes the orders and sends a confirmation message to the trader's browser.
 - This process is repeated continuously until the user doesn't log out.
 - This real-time feature massively helped the stock trading platform to gain more active users.
- **Real-Time Dashboards:** Real-time dashboards monitor and track data from sensors, databases, or other sources in real time. Websocket are very useful here as they enable the software to implement real-time dashboards that update automatically as the data changes. A simplified overview of Websocket implementation in real-time dashboards is as follows:
 - When a user opens a real-time dashboard, their browser establishes a Websocket connection to the dashboard's server.
 - The dashboard's server sends the user a snapshot of the dashboard data.
 - The user's browser displays the dashboard data to the user.
 - The dashboard's server sends updates to the user's browser whenever the data changes.
 - The user's browser updates the displayed dashboard data to reflect the changes.
 - This process is repeated continuously, allowing users to monitor and track data in real time.

6 Conclusion and Future Works

Websocket have been a welcome addition to the inventory of developers. They have enabled many different kinds of functionalities to be added to the programs and pushed the capabilities of web development forward.

This study examined the power of Websocket through a real-time chatting system, offering a picture of how Websockets and many such real-time bidirectional protocols are currently being used. Reflecting on the aim of the Websocket protocol, an assessment of major success can be proven in the field from an empirical perspective. Implementing Websockets for a real-time chatting system is just the tip of the iceberg of a protocol due to its powerful and versatile feature.

There is still room for improvement, for both the Websocket protocol and the real-time chatting system. While implementing Websocket, it should always be in mind to stick to the best practices in order to create safer, more stable, and secure applications. The Websocket protocol is a very powerful tool and should be handled as such, not with any malicious or questionable intent.

The real-time chatting system gets the job done very efficiently and effectively, yet it is not exactly at the position of being in the market to be deployed and used by the masses. Thus, the future goals of this system are to make it capable of competing in the market with other chatting systems. In conclusion, the real-time chatting system can be assumed to be a success to showcase and present the capabilities of Websocket.

References

1. Teixeira, P. (2012). *Professional Node.js: Building Javascript based scalable software*. Wiley.
2. Kwon, H., Kwon, Y., & Han, J. (2022). Backward graph construction and lowering in DL compiler for model training on AI accelerators. In *Proceedings of the 2022 19th international SoC design conference (ISOCC)* (pp. 91–92).
3. Luo, Z., Yan, S., & Luo, S. (2023). Multitask fine tuning on pretrained language model for retrieval-based question answering in automotive domain. *Mathematics*, 11, 2733.
4. Kumar, I., Sahu, S., Chakraborty, R., Mishra, S., & Chaudhary, V. (2023, February). A precise smart parking model with applied wireless sensor network for urban setting. In *International conference on innovative computing and communication* (pp. 227–235). Springer Nature Singapore.
5. Nandi, A., Yadav, S., Hobisyashi, A., Ghosh, A., Mishra, S., & Chaudhary, V. (2023, February). An ensemble learning approach for detection of COVID-19 using chest X-ray. In *International conference on innovative computing and communication* (pp. 237–244). Springer Nature Singapore.
6. Kim, J., Kong, J., & Son, J. (2021). *Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech*. [arXiv:2106.06103](https://arxiv.org/abs/2106.06103)
7. Heitkötter, H., Majchrzak, T. A., Ruland, B., & Weber, T. (2013). Evaluating frameworks for creating mobile web apps. In *Proceedings of the 9th international conference on web information systems and technologies (WEBIST 2013)* (pp. 209–221).
8. Dua, B., Kriti, K., Mishra, S., Shashidhar, C., dos Reis, M. C., & de Albuquerque, V. H. C. (2023, February). Applied computational intelligence for breast cancer detection. In *International conference on innovative computing and communication* (pp. 435–443). Springer Nature Singapore.
9. Alharbi, S., Alrazgan, M., Alrashed, A., Alnomasi, T., Almojel, R., Alharbi, R., Alharbi, S., Alturki, S., Alshehri, F., & Almojil, M. (2021). Automatic speech recognition: Systematic literature review. *IEEE Access*, 9, 131858–131876.
10. Meng, F., & Hyung, C. J. (2022). Research on multi-NPC marine game AI system based on Q-learning algorithm. In *Proceedings of the 2022 IEEE international conference on artificial intelligence and computer applications (ICAICA)* (pp. 648–652).
11. Rosa, A. S., & Shalahuddin, M. (2008). *Pemrograman J2ME Belajar Cepat Pemrograman Perangkat Telekomunikasi Mobile*. Informatika Publishing.
12. Patra, P., Ved, V., Chakraborty, S., Mishra, S., & Chaudhary, V. (2023, February). ECG-based cardiac abnormalities analysis using adaptive artificial neural network. In *International conference on innovative computing and communication* (pp. 245–251). Springer Nature Singapore.

13. Li, J., Deng, L., Haeb-Umbach, R., & Gong, Y. (Eds.). (2016). Chapter 11—Summary and future directions. In *A bridge to practical applications; robust automatic speech recognition* (pp. 261–280). Academic Press. ISBN: 9780128023983.
14. Goel, A. K., Pengoria, A., Kumar, A., & Agrawal, K. K. (2022). Composite movie recommendation system. In *Proceedings of the 2022 8th international conference on advanced computing and communication systems (ICACCS 2022)* (pp. 1273–1278).
15. Malik, A. A., & Brem, A. (2021). Digital twins for collaborative robots: A case study in human-robot interaction. *Robotics Computer-Integration Manufacturing*, 68, 102092.
16. Verma, S., Sinha, S., Chaudhury, P., Mishra, S., & Alkhayyat, A. (2023). Crop yield forecasting with precise machine learning. In Hassanien, A. E., Castillo, O., Anand, S., Jaiswal, A. (Eds.), *International conference on innovative computing and communications* (Vol. 537). ICICC 2023. Lecture notes in networks and systems. Springer. https://doi.org/10.1007/978-981-99-3010-4_38